CITY UNIVERSITY MALAYSIA

Cyberjaya Campus — Faculty of Information Technology

BIT2083 Fundamental of Computational Thinking: Python

Final Project Report (40%)

15-Minute Neighborhood Score

A Console Application for SDG 11: Sustainable Cities and Communities

Full Name	[Your Full Name]
Student ID	[Your Student ID]
Class Code	[Your Class Code]
Program	[Your Program]
GitHub Repository	[https://github.com/<your-username>/15-minute-neighborhood-score]

August 2026

Note: replace the bracketed placeholders above with your own details (and each group member's) before submission, per the assignment instructions. NRIC/passport numbers are not included in this document — enter that information directly wherever your submission process requires it.

Contents

1. Introduction & SDG Background
2. Problem Statement
3. System Objectives
4. Application Design
5. Implementation Details
6. Testing & Sample Outputs
7. Discussion & Limitations
8. Conclusion

1. Introduction & SDG Background

Cities are where the sustainability challenge is most visible. As urban populations grow, so does reliance on private vehicles for even the shortest everyday trips — buying groceries, visiting a clinic, or dropping a child at school. This car-dependent pattern of urban design increases greenhouse gas emissions, congestion, and air pollution, while also reducing physical activity and excluding residents who cannot drive, such as children, older adults, and lower-income households without a car.

This project addresses Sustainable Development Goal 11: Sustainable Cities and Communities, which calls for inclusive, safe, resilient, and sustainable human settlements. Two SDG 11 targets are especially relevant here: Target 11.2, on providing access to safe, affordable, and accessible transport systems, and Target 11.7, on universal access to safe, inclusive, and accessible green and public spaces. Both targets are fundamentally about proximity — whether the things people need in daily life are close enough to reach without a car.

That proximity idea is the basis of the “15-minute city” concept in contemporary urban planning: a neighborhood is considered well designed when a resident can reach most daily essentials — food, healthcare, schooling, transit, green space, and everyday services — within roughly a 15-minute walk from home. This project operationalizes that idea as a lightweight, practical tool: the 15-Minute Neighborhood Score, a Python console application that lets a user quantify how well their own neighborhood (or a neighborhood they are considering moving to) meets this standard, and receive concrete suggestions for what is missing.

2. Problem Statement

Whether a neighborhood is walkable is rarely obvious from the inside. Residents can usually name the nearest grocery store or clinic, but they seldom have a single, objective number that summarizes accessibility across every category of daily need at once — and they have no simple way to compare that number between two locations when deciding where to live. Existing walkability tools tend to be either professional GIS software aimed at urban planners, or web services that require an internet connection, an account, and access to a live mapping API — none of which are available to every prospective tenant sitting down to compare two apartment listings.

The concrete problem this project solves is therefore: how can an individual resident, using nothing more than a basic Python environment, record what they already know about a neighborhood (roughly how long it takes to walk to the nearest grocery store, clinic, school, transit stop, park, and everyday service), and turn that into an objective, comparable walkability score with actionable feedback — supporting a more sustainable, less car-dependent housing decision.

3. System Objectives

The 15-Minute Neighborhood Score application is designed to meet the following objectives:

- Allow a user to record a named “neighborhood assessment” capturing estimated one-way walking time to the nearest instance of six essential amenity categories.
- Compute an objective, weighted 0–100 walkability score from that data, and classify it into a clear rating band (Excellent / Good / Fair / Poor).
- Persist every assessment to disk (JSON) so a user's history survives across multiple runs of the program.
- Generate specific, targeted recommendations that identify the weakest-scoring categories and suggest realistic ways to address them.
- Support direct, side-by-side comparison of two saved assessments, to help a user decide between two housing options.
- Present all of this through a clear, validated, menu-driven console interface that never crashes on bad input.
- Demonstrate computational thinking (decomposition, abstraction, pattern recognition, and algorithmic thinking) and modular, well-organized Python programming throughout the implementation.

4. Application Design

The application follows a layered architecture: a presentation layer (ui.py) handles all console formatting; a controller layer (app.py) runs the menu loop and dispatches to one handler function per menu option; a domain layer (models.py, scoring.py, recommendations.py) holds the actual business logic; and a persistence layer (storage.py) handles reading and writing the JSON data file. Cross-cutting utilities (validation.py, constants.py) are used by every other layer. No layer reaches past its immediate neighbor — for example, ui.py never touches the JSON file, and storage.py never prints to the console — which keeps each module independently testable and readable.

4.1 Program Flowchart

The left panel below traces the main program loop: load any saved data, repeatedly display the menu, validate the user's choice, and either dispatch to the matching handler or exit. The right panel zooms into the most representative handler, “Create New Assessment,” showing the nested loop structure (a for-loop over the six amenity categories, each with an inner while-retry validation loop) that recurs throughout the application.

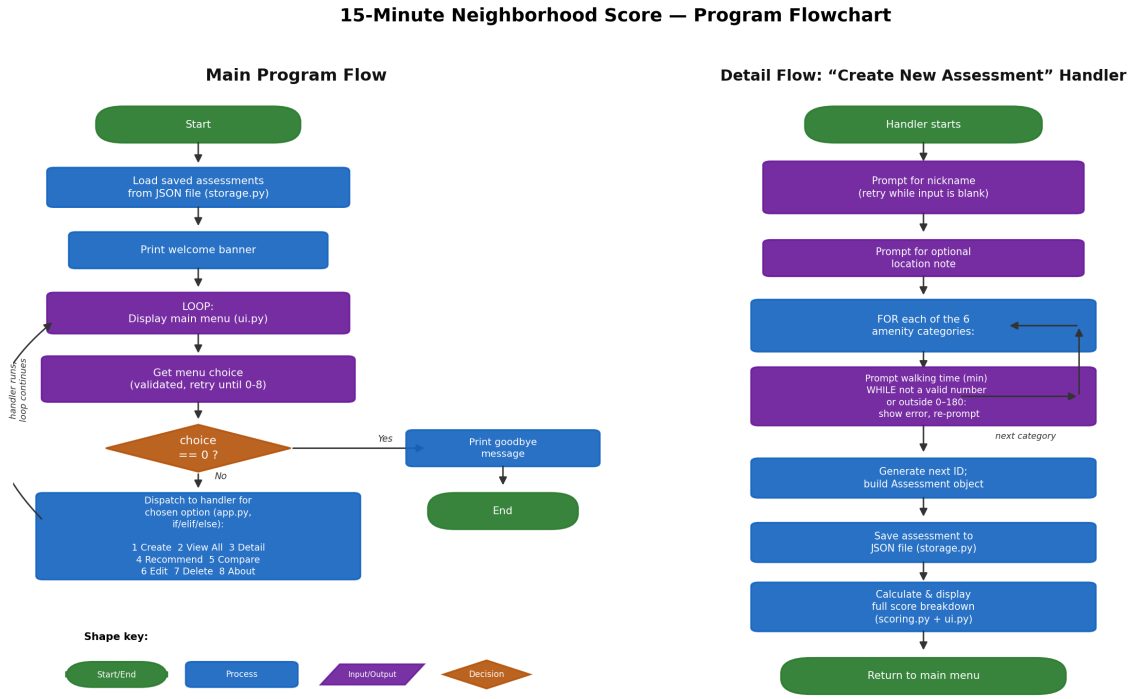
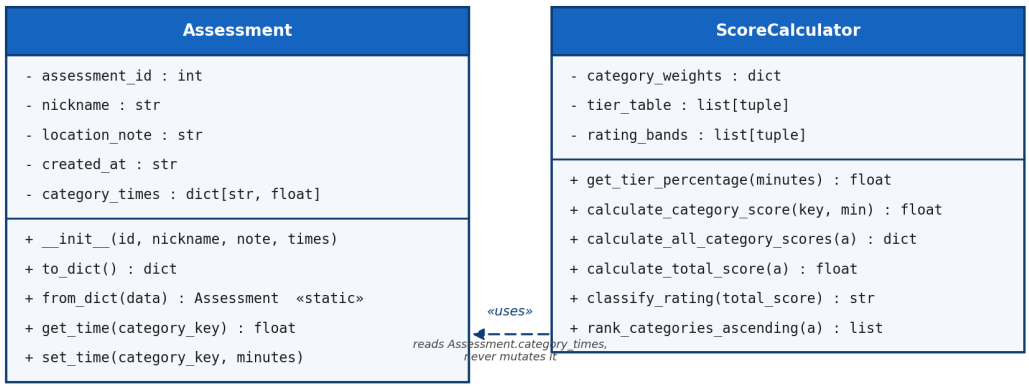


Figure 1. Program flowchart — main loop (left) and Create New Assessment detail flow (right).

4.2 UML Class Diagram

Two classes anchor the design. Assessment (models.py) is a pure data entity holding one neighborhood's raw input. ScoreCalculator (scoring.py) is a pure behavior entity that turns any Assessment into scores, a rating, and a weakest-category ranking. They are kept deliberately separate — data from behavior — so neither class becomes a “God object” responsible for both storing information and computing with it. Every other responsibility in the system (validation, persistence, recommendations, console output, menu control flow) is implemented as plain module-level functions rather than additional classes, in line with the assignment's explicit requirement for user-defined functions as the primary building block.

UML Class Diagram — 15-Minute Neighborhood Score



Design notes:

- Two classes only, each earning its place independently — data (Assessment) is kept separate from behavior (ScoreCalculator) to avoid one “God object” doing both jobs.
- Assessment never caches a total score or rating — both are always re-derived by ScoreCalculator on demand, so there is exactly one source of truth (no risk of a cached score going stale after an edit).
- All other application logic (validation, storage/persistence, recommendations, console presentation, menu control flow) lives in plain, single-purpose module-level functions in validation.py, storage.py, recommendations.py, ui.py and app.py — see the Implementation Details section for the full function inventory.

Figure 2. UML class diagram for Assessment and ScoreCalculator.

5. Implementation Details

The application is implemented as a proper Python package (neighborhood_score/) with nine files and 54 named functions/methods in total — well beyond the assignment's minimum of five user-defined functions — each with a single, clearly named responsibility. The table below summarizes each module.

Module	Responsibility	Fns
constants.py	Shared configuration: category weights, tier table, rating bands, file paths	1
validation.py	Reusable retry-loop input validation (float/int range, menu choice, yes/no)	6
models.py	Assessment class — the data entity for one neighborhood	5
scoring.py	ScoreCalculator class — the scoring algorithm — plus a factory function	8
storage.py	JSON persistence and CRUD operations (load, save, add, find, update, delete)	7
recommendations.py	Weak-category detection and canned improvement tips	3
ui.py	All console formatting/printing — presentation only, no business logic	12
app.py	Menu loop and one handler function per menu option	11
main.py	Entry point (2 lines: calls app.run())	1

5.1 The Scoring Algorithm

Each of the six amenity categories (Grocery, Healthcare, Education, Transit, Parks, Retail) carries a weight that sums to 100. A single function, get_tier_percentage(minutes), converts a walking time into a tier percentage using one reusable piecewise rule, applied identically to every category rather than duplicated six times:

Walking time	Tier percentage earned
0 – 5 minutes	100%
> 5 – 10 minutes	75%
> 10 – 15 minutes	50%
> 15 – 25 minutes	20%
> 25 minutes	0%

The total score is calculate_total_score(assessment) = Σ (category weight × tier percentage), producing a value from 0 to 100, which classify_rating() then maps to Excellent (≥85), Good (70–84), Fair (50–69), or Poor (<50).

5.2 Computational Thinking in This Design

- **Decomposition** — the problem is split into 8 single-responsibility modules (validation, storage, scoring, recommendations, models, ui, app, constants), each further broken into small functions such as prompt_float_in_range() or calculate_category_score().
- **Abstraction** — ScoreCalculator hides the tier lookup, weighting, and summation behind three simple calls (calculate_total_score, calculate_all_category_scores, classify_rating); calling code never needs to know how the tier table is structured to use it.
- **Pattern recognition** — get_tier_percentage() is written once and applied identically across all six categories instead of being duplicated six times, and the same “loop until valid” shape is reused for every console input via prompt_float_in_range(), prompt_int_in_range(), and prompt_menu_choice().
- **Algorithmic thinking** — calculate_total_score() is an explicit accumulate-over-a-collection algorithm (iterate categories → look up tier → multiply by weight → sum), and rank_categories_ascending() is an explicit sort-then-slice algorithm used to surface the weakest categories for recommendations.

5.3 Validation & Error Handling

Every input() call in the application routes through one of six functions in validation.py, so no validation logic is duplicated ad hoc elsewhere. Menu choices, nicknames, and walking times are all validated with a retry loop that re-prompts with a specific error message rather than crashing — for example, non-numeric text, negative numbers, and unrealistically large travel times (over 180 minutes) are all rejected with a distinct, actionable message. Persistence is similarly defensive: a missing data file is treated as an empty history, and a corrupted data file is renamed to a .bak backup and reported to the user, rather than crashing the application on launch.

6. Testing & Sample Outputs

The application was tested with a scripted driver that feeds a long, deliberate sequence of console input into the real program (not a mock) and captures its actual output, so every transcript in this section and in tests/manual_test_transcript.txt is genuine program output. Three test runs were performed: an empty first-run session, a corrupted-data-file recovery session, and one continuous 18-scenario walkthrough covering assessment creation, tier boundaries, recommendations, comparison, editing, deletion, and persistence. All runs completed with exit code 0 and zero unhandled exceptions.

6.1 Test Results Summary

Scenario	Expected Result	Result
Empty history on first run	Graceful message, no crash	Pass
Corrupted JSON data file	Renamed to .bak, empty history, no crash	Pass
Invalid menu input ("9", "-1", "abc", "")	Rejected + re-prompted each time	Pass
Non-numeric / negative / oversized minutes	Rejected with specific message	Pass
All categories ≤ 5 min	Total = 100.0, Rating = Excellent	Pass
All categories > 25 min	Total = 0.0, Rating = Poor	Pass
Tier-boundary values (5/6/10/11/15/16/25/26)	Correct tier assigned at every boundary	Pass
Recommendations on a perfect-score assessment	Graceful “no major gaps” message	Pass
Compare two different assessments	Correct side-by-side table + verdict	Pass
Compare an assessment against itself	Rejected with a clear error	Pass
Edit a category time / rename a nickname	Score and display update correctly	Pass
Delete: cancel, then confirm	Cancelled first, removed second; ID gap visible	Pass
Persistence across a fresh relaunch	All data, including edits, survived	Pass
48-character nickname in list/compare views	Truncated cleanly, no column overlap	Pass

6.2 Sample Transcript — Input Validation & Score Breakdown

```
Select an option (0-8): [ERROR] '9' is not a valid option. Please choose one of: 0, 1, ...
8.
Select an option (0-8): [ERROR] '-1' is not a valid option. Please choose one of: 0, 1,
... 8.
Select an option (0-8): [ERROR] 'abc' is not a valid option. Please choose one of: 0, 1,
... 8.
Select an option (0-8): [ERROR] '' is not a valid option. Please choose one of: 0, 1, ...
8.
Select an option (0-8): 1

------------------------------------------------------------
NEW NEIGHBORHOOD ASSESSMENT
------------------------------------------------------------
Enter a nickname for this neighborhood: [ERROR] This field cannot be empty.
Enter a nickname for this neighborhood: Current Apartment
Enter an optional location note (or press Enter to skip): Jalan Ampang, KL

Grocery / Fresh Food Access (minutes): [ERROR] 'abc' is not a valid number.
Grocery / Fresh Food Access (minutes): [ERROR] Value cannot be negative.
Grocery / Fresh Food Access (minutes): 4
Healthcare / Pharmacy (minutes): 12
Education / Childcare (minutes): 20
Public Transit Access (minutes): 3
Parks / Green Space (minutes): 9
Everyday Retail & Services (minutes): 7

[OK] Assessment #1 "Current Apartment" saved.
------------------------------------------------------------
SCORE BREAKDOWN: Current Apartment  (ID 1)
------------------------------------------------------------
Category                       Minutes   Tier      Points
------------------------------------------------------------
Grocery / Fresh Food Access        4.0   100%   20.0 / 20
Healthcare / Pharmacy             12.0    50%    7.5 / 15
Education / Childcare             20.0    20%    3.0 / 15
Public Transit Access              3.0   100%   20.0 / 20
Parks / Green Space                9.0    75%   11.3 / 15
Everyday Retail & Services         7.0    75%   11.3 / 15
------------------------------------------------------------
TOTAL SCORE: 73.0 / 100   ->   Rating: GOOD
------------------------------------------------------------
```

Excerpt from tests/manual_test_transcript.txt — invalid-input retries and a full score breakdown.

6.3 Sample Transcript — Corrupted Data File Recovery

```
Setup: data/assessments.json manually overwritten with invalid JSON before launch.

Select an option (0-8): [!] The saved data file was unreadable and has been moved
to '.../data/assessments.json.bak'. Starting with an empty history.
============================================================
                      ALL ASSESSMENTS
============================================================
No assessments yet. Choose option 1 to create your first one.
```

Excerpt showing graceful recovery from a corrupted JSON data file.

6.4 Sample Transcript — Recommendations & Compare Validation

```
============================================================
                      RECOMMENDATIONS
============================================================
- Education / Childcare: Schools or childcare are far away. Carpooling groups
  or petitioning for a satellite childcare centre may help.
- Healthcare / Pharmacy: No nearby clinic or pharmacy. Look into telehealth
  options, or raise this gap with local health authorities.
------------------------------------------------------------
...
Enter ID of first assessment: 1
Enter ID of second assessment: 1
[ERROR] Please choose two different assessments to compare.
```

Excerpt showing generated recommendations and rejection of an invalid comparison.

6.5 A Bug Found and Fixed During Testing

Testing was not purely confirmatory — it surfaced a real defect. The first version of the “Compare Two Assessments” screen embedded each assessment's nickname directly into a fixed-width table column header. Python's string formatting only pads short strings; it does not truncate long ones, so a nickname such as “Perfect Score Home (ID 2)” (26 characters) overran its 15-character column and ran into the next column with no separating space. The fix, verified against a deliberately long 48-character nickname, replaced the embedded nickname with fixed-width “Option A” / “Option B” column headers (with the full names shown in a legend above the table), and added a general-purpose truncate_text() helper applied defensively to the history list and score-breakdown views as well.

7. Discussion & Limitations

By turning “is my neighborhood walkable?” into a concrete, weighted, comparable number, the application gives residents a practical tool for a more sustainable housing decision — directly supporting SDG 11's aim of accessible, inclusive urban living. The comparison feature in particular addresses a real decision point: choosing between two homes with an explicit accessibility trade-off in view, rather than an unquantified gut feeling. The recommendations feature turns the score into action, pointing at specific categories worth advocating for rather than leaving the user with a number and nothing to do with it.

The current implementation has some acknowledged limitations:

- **Self-reported walking times** — the tool relies on the user's own estimate of travel time rather than a real routing engine, so accuracy depends on the user's familiarity with the area.
- **No mapping integration** — a production version could call a mapping API (e.g. OpenStreetMap routing) to compute true walking distances automatically instead of asking the user to estimate them.
- **Fixed category weights** — the six category weights are the same for every user, though needs genuinely differ (a household with young children may weight Education far higher than a retiree would).
- **Presence, not quality** — the model only measures whether an amenity is nearby in time, not whether it is safe, well-lit, or pleasant to walk to.
- **Single-user local storage** — assessments are stored in one local JSON file, which is sufficient for an individual but would need a proper database and accounts to support multiple concurrent users.

These are natural directions for future work, but none of them undermine the core contribution: a working, tested, zero-dependency console tool that turns a vague sense of “this area feels walkable” into an objective, actionable, and comparable score.

8. Conclusion

This project set out to apply computational thinking and Python programming to a real sustainability problem under SDG 11: helping residents understand and compare how accessible their neighborhood actually is. The result, 15-Minute Neighborhood Score, is a fully functional, menu-driven console application built from 54 single-responsibility functions and methods across nine files and two classes, with robust input validation, persistent JSON storage, and a tested, documented codebase. Beyond meeting the assignment's technical requirements, the application demonstrates a genuine, defensible design process — including a real bug found and fixed through deliberate testing — and a concrete, practical answer to a real question a person might actually ask before choosing where to live.